Lab 0.
Individual assignment

1. Individual assignment topic

Develop a modern, responsive, and accessible web application using HTML, CSS, JavaScript/TypeScript, and a REST API.

2. Goal

Develop, test, and publish your own web application, demonstrating practical proficiency with the discipline's core technologies: HTML5, CSS, JavaScript, TypeScript, REST APIs, and modern frontend development tools.

3. Core requirements

Meet the following requirements:

- choose a subject area independently and agree on it with your instructor;
- create a semantic HTML5 structure;
- implement modern CSS styling and responsive design;
- meet basic web accessibility requirements in line with WCAG 2.2;
- add interactivity with JavaScript;
- integrate at least one REST API;
- implement loading and error states;
- use TypeScript to add types to part of your application;
- use a modular project structure and Vite/NPM;
- debug, test, and review your code;
- publish your completed web application online;
- provide a Git repository, README, and materials for your defense.

Web application format

Build a web application with a simple structure in one of these formats:

- a landing page with interactive elements and data from an API; or

- a simple MPA with 2–3 pages, for example: home → catalog → details.

You are not required to implement:

- user authentication or registration;
- your own backend;
- a database;
- a complex administration system;
- full CRUD functionality;
- complex business logic.

The main focus is frontend development, working with APIs, code quality, accessibility, and responsive design.

4. Examples of subject areas and web application topics

Choose one of the suggested topics or propose your own and agree on it with your instructor.

No.	Subject area / topic	Public API	Suggested format
1	**Book catalog** — search for books and view information about them	Open Library API — a public API with searchable information about books, authors, and editions, including book covers	MPA: catalog → details
2	**TV series catalog** — search for series and view information about them	TVmaze API — a REST API for searching shows and episodes, and viewing cast and rating information	MPA: search → list → details
3	**Recipe catalog** — search, filter, and browse recipes	DummyJSON Recipes API — a ready-made dataset of recipes with search, filtering, sorting, and pagination	Landing page + catalog or MPA
4	**Travel weather forecast service** — get a forecast for a selected city	Open-Meteo API — weather forecast and geocoding APIs with no API key required	Landing page: city search → forecast
5	**Dog breed catalog** — browse breeds and a photo gallery	Dog API — a public API with a list of breeds, breed search, and images	Landing page + gallery / MPA
6	**Pokémon catalog** — browse characters and view detailed information	PokéAPI — a free, public REST API with information about Pokémon, types, abilities, and sprites	MPA: catalog → details

No.	Subject area / topic	Public API	Suggested format
7	**Virtual art gallery** — search for and browse works of art	Art Institute of Chicago API — a public REST-style JSON API with information about works of art	Landing page + gallery + details
8	**Nobel laureate catalog** — search for and view information about laureates and prizes	Nobel Prize API — a public API with information about laureates, prizes, and related entities	MPA: list → details
9	**Currency converter** — compare and convert currencies	Frankfurter API — an API for current and historical exchange rates	Landing page
10	**Space launch and event catalog** — browse upcoming launches	Launch Library 2 API — a public API with information about launches, events, missions, and spacecraft	Landing page + launch list + details

Topic selection recommendations

Choose a topic that lets you demonstrate:

- retrieving data through a REST API;
- search, filtering, or sorting;
- displaying a list of data;
- viewing detailed information;
- at least 3 interactive scenarios;
- loading and error states;
- a responsive and accessible interface.

5. Completion stages and time allocation

No.	Stage	Hours
1	Subject area analysis, project structure, semantic HTML5	5
2	CSS, box model, cascade, specificity, CSS variables	5
3	SEO, web accessibility, responsive design	8
4	JavaScript, DOM, events, modules, debugging	9
5	REST APIs, Fetch/async-await, loading and error states	9
6	TypeScript and typing APIs/code	8
7	Typing existing code, refactoring, and code review	6
8	Vite/NPM, Git, ESLint, Prettier, testing, deployment	6
9	Comprehensive improvements and defense preparation	6

No.	Stage	Hours
	Total	**62**

6. Deliverables

You must submit:

1. A Git repository with the source code.
2. A published web application with an accessible URL.
3. A README describing the project's purpose, technologies, API, setup instructions, and main features, plus a brief technical report describing the solutions you implemented.
4. A defense of your work, including a demonstration of the application and an explanation of your code.

7. Minimum requirements for the web application

- include a semantic HTML5 structure;
- have a responsive interface;
- use CSS variables and modern CSS layout techniques;
- include at least 3 interactive scenarios;
- retrieve and display data from a REST API;
- handle loading and error states;
- include TypeScript code;
- have basic tests;
- pass a basic accessibility check;
- be published online.

8. Defense

During your defense:

- demonstrate your web application;
- show your Git repository;
- explain your project structure;
- explain key parts of your HTML, CSS, and JavaScript/TypeScript;
- demonstrate how the REST API works;

- demonstrate debugging and testing;
- justify your main technical decisions;
- answer questions about your own code.

9. Grading

Criterion	Points
HTML, semantics, and structure	10
CSS, layout, and responsive design	15
SEO and web accessibility	10
JavaScript, DOM, and events	15
REST APIs and asynchronous JavaScript	15
TypeScript	10
Testing, debugging, and code quality	20
Deployment, README, and project presentation	5
Total	**100**

> **Required condition for a passing grade.** To pass, you must submit a working project, a Git repository, and a published version of your web application, and demonstrate that you can explain your code and the technical decisions you made.

10. Submission

Send your work by email to denys.matsevych@oa.edu.ua.

Email subject:

```
[Web] Individual assignment
```

Include:

- a link to your Git repository;
- a link to your published web application.